

UNIVERSIDAD FIDÉLITAS



Programación Avanzada

PEDIDOS360

Sistema Web de Gestión de Pedidos

Proyecto Informe Técnico Final

Estudiantes:

Justin José Esquivel Marchena

Keylor Mora Chacón

David Montero Hernández

José Adrián Sancho Astacio

Profesor: Raul Alexander Monge Mora

Período: 2026 - I Cuatrimestre

Tabla de contenido

1.INTRODUCCIÓN	3
1.1 Propósito del sistema.....	3
1.2 Alcance	3
1.3 Tecnologías utilizadas	4
2.ARQUITECTURA DEL SISTEMA	4
2.1 Patrón de diseño (MVC).....	4
2.2 Estructura de capas	5
2.3 API REST interna.....	6
3.MODELO DE DATOS	7
3.1 Diagrama entidad-relación	7
3.2 Descripción de entidades	8
<i>AspNetUsers / ApplicationUser</i>	8
<i>CATEGORIA</i>	8
<i>PRODUCTO</i>	8
<i>CLIENTE</i>	8
<i>CLIENTE_DIRECCION</i>	8
<i>PEDIDO</i>	8
<i>PEDIDO_DETALLE</i>	8
3.3 Restricciones de integridad.....	9
4.DECISIONES DE DISEÑO	10
4.1 Autenticación y autorización basada en roles.....	10
4.2 Cálculo de totales con impuestos por línea	10
4.3 Flujo de estados del pedido	10
4.4 Gestión de stock en tiempo real.....	11
4.5 Exportación de documentos (PDF y Excel)	11
5.EVIDENCIAS DE REQUISITOS IMPLEMENTADOS	11
6.CONCLUSIONES	15

1.INTRODUCCIÓN

1.1 Propósito del sistema

Pedidos360 es una aplicación web empresarial desarrollada para la gestión integral del ciclo de vida de pedidos comerciales. El sistema permite a una organización registrar clientes, mantener un catálogo de productos con control de inventario, crear y administrar pedidos multi-línea con descuentos por artículo e impuestos diferenciados, así como emitir comprobantes en formato PDF y reportes en hoja de cálculo (Excel). El propósito académico del proyecto es demostrar la aplicación práctica de los conceptos de bases de datos relacionales, diseño de esquemas normalizados, acceso a datos mediante un ORM y la implementación de endpoints de consulta mediante una API REST interna.

1.2 Alcance

El sistema abarca los siguientes módulos funcionales:

- **Autenticación y gestión de usuarios:** registro, inicio de sesión y administración de cuentas bajo tres roles (Administrador, Ventas y Operaciones).
- **Catálogo de productos:** creación, edición, desactivación y búsqueda de productos con categoría, precio, porcentaje de impuesto y stock.
- **Gestión de clientes:** CRUD de clientes con datos personales y múltiples direcciones.
- **Gestión de pedidos:** creación de pedidos con líneas de detalle, cálculo automático de subtotales, impuestos y total; flujo de estados y exportación.
- **Dashboard de indicadores:** métricas en tiempo real sobre pedidos, clientes y productos.
- **API REST interna:** endpoints de búsqueda y cálculo consumidos por la interfaz vía AJAX.

1.3 Tecnologías utilizadas

Componente	Tecnología / Biblioteca	Versión
Lenguaje de programación	C# (.NET 8)	8.0
Framework web	ASP.NET Core MVC	8.0
ORM	Entity Framework Core	8.x
Motor de base de datos	Microsoft SQL Server Express	2022
Autenticación	ASP.NET Core Identity	8.0
Generación de PDF	iText 7 (iTextSharp)	7.x
Generación de Excel	EPPlus	6.x
Interfaz de usuario	Bootstrap 5 + Razor Views	5.3
IDE / Entorno de desarrollo	Visual Studio 2022	17.x
Control de versiones	Git / GitHub	—

2.ARQUITECTURA DEL SISTEMA

2.1 Patrón de diseño (MVC)

El sistema implementa el patrón Modelo-Vista-Controlador (MVC) provisto por el framework ASP.NET Core. Esta separación de responsabilidades permite que la lógica de negocio, la presentación y el acceso a datos se mantengan desacoplados, facilitando el mantenimiento y la escalabilidad de la solución.

- **Modelo:** clases ubicadas en la carpeta *Models/*, anotadas con Data Annotations para validación y mapeo al esquema relacional.

- **Vista:** archivos *.cshtml* (Razor) organizados por controlador en *Views/*; se reutiliza un *_Layout.cshtml* compartido para cabecera, navegación y pie de página.
- **Controlador:** clases en *Controllers/* que reciben las solicitudes HTTP, interactúan con el contexto de datos y devuelven la vista o una respuesta JSON.

2.2 Estructura de capas

La organización de carpetas del proyecto refleja una separación lógica en capas:

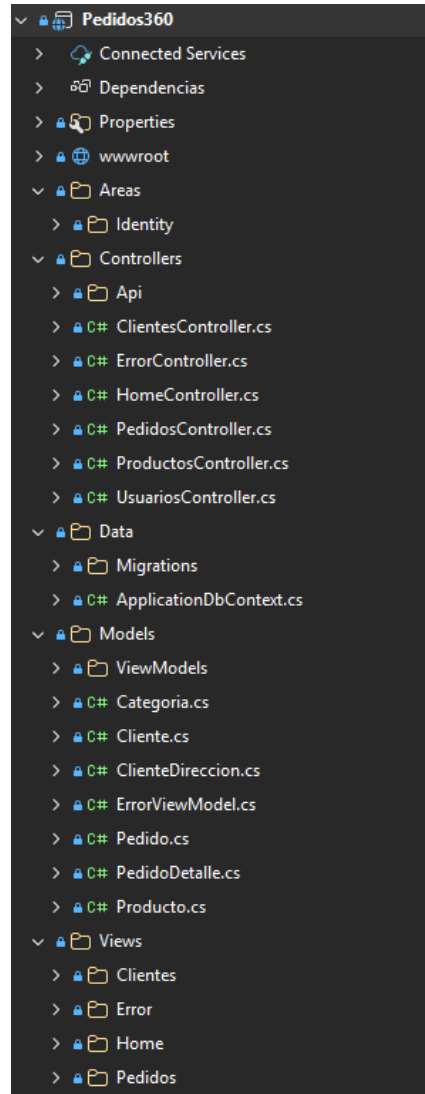


Figura 1. Estructura de directorios del proyecto.

2.3 API REST interna

Además del flujo MVC tradicional, el sistema expone tres endpoints JSON bajo el prefijo */api/* que son consumidos de forma asíncrona (AJAX) desde las vistas Razor:

Endpoint	Método	Descripción	Roles permitidos
GET <code>/api/productos/buscar?q={texto}</code>	GET	Búsqueda incremental de productos activos (autocomplete). Retorna id, nombre, precio, impuesto y stock.	Autenticado
POST <code>/api/pedidos/calcular</code>	POST	Recibe lista de líneas (productoId, cantidad, descuento) y retorna subtotal, impuestos y total calculados en el servidor.	Autenticado
POST <code>/api/categorias</code>	POST	Crea una nueva categoría en tiempo real desde el formulario de productos sin recargar la página.	Admin, Ventas

El cálculo de totales en el servidor (en lugar del cliente) garantiza que los valores guardados en la base de datos sean siempre consistentes con la lógica de negocio, y no manipulables desde el navegador.

3.MODELO DE DATOS

3.1 Diagrama entidad-relación

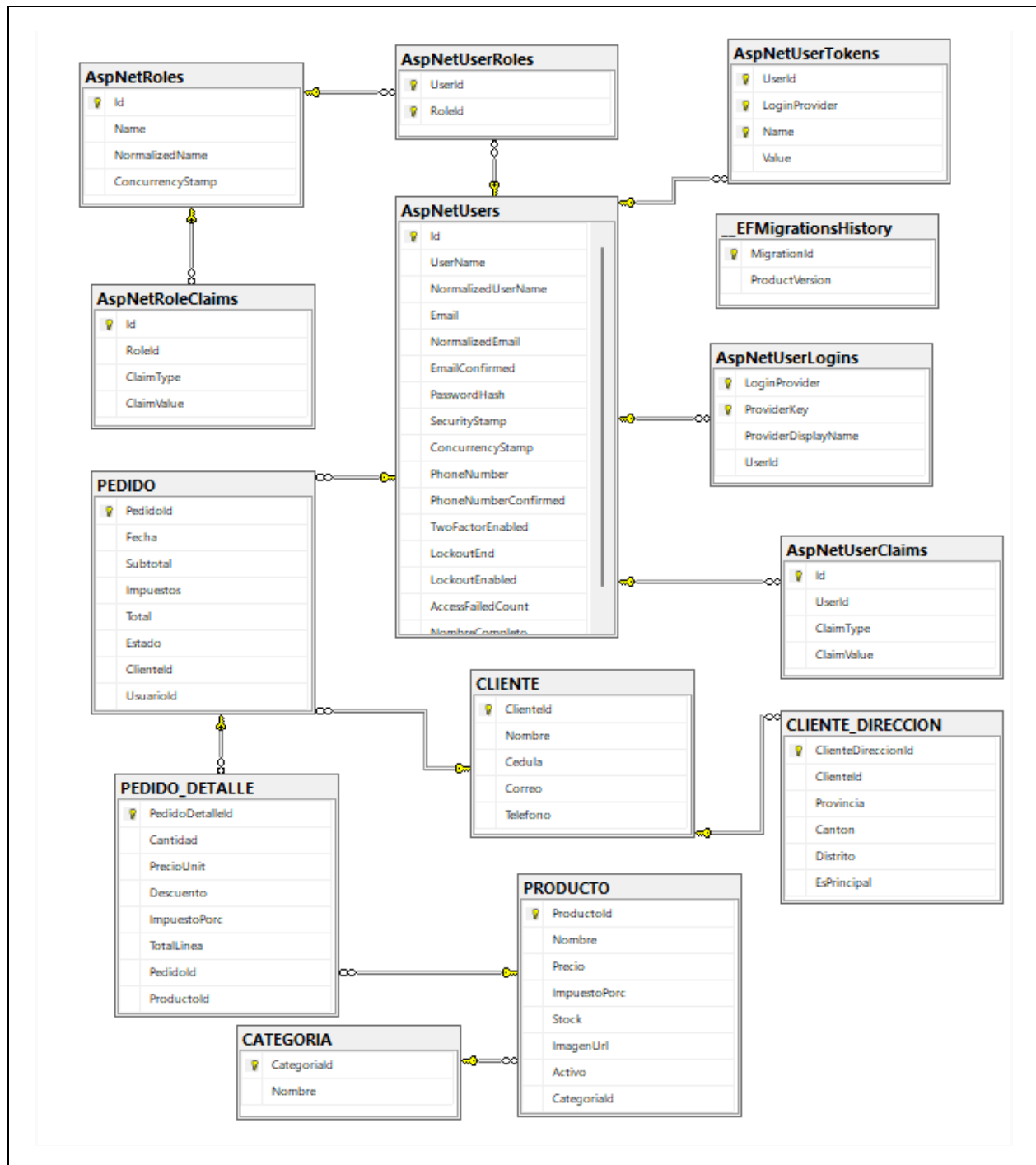


Figura 2. Diagrama entidad-relación del esquema Pedidos360DB.

3.2 Descripción de entidades

AspNetUsers / ApplicationUser

Extiende la tabla estándar de ASP.NET Core Identity con el campo *NombreCompleto* (*nvarchar 100*). Almacena las credenciales hasheadas, el estado de bloqueo y la confirmación de correo. Los roles (Admin, Ventas, Operaciones) se gestionan mediante las tablas *AspNetRoles* y *AspNetUserRoles*.

CATEGORIA

Agrupar productos bajo una denominación común (máximo 100 caracteres). Actúa como catálogo maestro de clasificación del inventario.

PRODUCTO

Representa un artículo del catálogo. Contiene el precio unitario base, el porcentaje de impuesto (IVA) asociado al producto, la cantidad en stock y una imagen opcional codificada en Base64. El campo *Activo* permite retirar productos del catálogo sin eliminarlos, preservando la integridad de pedidos históricos.

CLIENTE

Almacena los datos de contacto del comprador. Los campos *Cedula* y *Correo* poseen índices únicos en la base de datos para evitar duplicados. La cédula sigue el patrón costarricense de nueve dígitos iniciando con número de provincia (1-7).

CLIENTE_DIRECCION

Permite registrar múltiples domicilios por cliente (provincia, cantón, distrito), con la posibilidad de marcar uno como principal. La relación con CLIENTE es de cascada en eliminación: al borrar un cliente se borran automáticamente sus direcciones.

PEDIDO

Encabezado del pedido. Almacena los totales calculados (subtotal sin IVA, monto de impuestos, total final) y el estado del ciclo de vida: **Pendiente a Confirmado a Facturado**. Referencia al cliente y al usuario (vendedor) que lo generó. La eliminación del cliente o del usuario no elimina pedidos en cascada (restricción *onDelete: Restrict*).

PEDIDO_DETALLE

Línea individual dentro de un pedido. Registra la cantidad, el precio unitario *vigente al momento de la venta*, el descuento porcentual aplicado, el porcentaje de impuesto del producto y el total calculado de la línea. Al eliminar un pedido, sus detalles se eliminan en cascada. La relación con PRODUCTO tiene restricción para no eliminar productos con pedidos asociados.

3.3 Restricciones de integridad

Entidad	Restricción	Tipo
CLIENTE.Cedula	Valor único en toda la tabla	Índice único (IX)
CLIENTE.Correo	Valor único en toda la tabla	Índice único (IX)
PEDIDO a CLIENTE	No permite eliminar cliente con pedidos	FK Restrict
PEDIDO a AspNetUsers	No permite eliminar usuario con pedidos	FK Restrict
PEDIDO_DETALLE a PEDIDO	Elimina detalles al eliminar el pedido	FK Cascade
PEDIDO_DETALLE a PRODUCTO	No permite eliminar producto con pedidos	FK Restrict
PRODUCTO a CATEGORIA	No permite eliminar categoría con productos	FK Restrict
CLIENTE_DIRECCION a CLIENTE	Elimina direcciones al eliminar el cliente	FK Cascade
PRODUCTO.Stock	No puede ser negativo; se valida en aplicación	Range [0, ∞)
PEDIDO_DETALLE.Descuento	Entre 0 y 100 (%)	Range [0,100]
PRODUCTO.ImpuestoPorc	Entre 0 y 100 (%)	Range [0,100]

4.DECISIONES DE DISEÑO

4.1 Autenticación y autorización basada en roles

Se optó por ASP.NET Core Identity como capa de autenticación por su integración nativa con Entity Framework Core y su soporte de roles, hashing seguro (PBKDF2) y protección contra ataques de fuerza bruta mediante bloqueo de cuenta. Se definieron tres roles con distintos niveles de acceso:

Rol	Módulos accesibles	Restricciones
Admin	Todos los módulos	Puede eliminar registros, cambiar estado a Facturado, gestionar usuarios
Ventas	Pedidos, Productos, Clientes (solo lectura/creación), Dashboard	No puede eliminar; no puede gestionar usuarios; no puede facturar
Operaciones	Pedidos (solo lectura), Productos (lectura/edición), Dashboard	No puede crear ni eliminar pedidos ni clientes

La aplicación inicializa los roles y usuarios predeterminados mediante la clase *SeedData*, ejecutada al arrancar en *Program.cs*. Esto garantiza que el sistema siempre cuente con al menos un administrador funcional en cualquier entorno de despliegue.

4.2 Cálculo de totales con impuestos por línea

El diseño contempla impuestos diferenciados por producto. Cada línea de detalle almacena el porcentaje de impuesto vigente al momento de la transacción, de modo que cambios futuros en la tarifa no afecten pedidos históricos. La fórmula aplicada es la siguiente:

El cálculo se realiza dos veces: una en tiempo real vía el endpoint `POST /api/pedidos/calcular` (para retroalimentación visual al usuario) y nuevamente en el servidor al momento de guardar el pedido en la base de datos, lo que elimina la posibilidad de manipulación desde el navegador.

4.3 Flujo de estados del pedido

Los pedidos siguen una máquina de estados lineal con transiciones controladas por rol, evitando retrocesos que pondrían en riesgo la consistencia del inventario y de los totales facturados:

Solo los pedidos en estado **Pendiente** pueden editarse. Una vez confirmados o facturados, el pedido queda inmutable. Esta restricción se implementa tanto en el controlador (retorna error si el estado no es Pendiente al intentar editar) como a nivel de interfaz (oculta el botón de edición condicionalmente).

4.4 Gestión de stock en tiempo real

Al crear un pedido, el stock de cada producto implicado se reduce inmediatamente en la misma transacción de base de datos que guarda el pedido y sus detalles. Al editar un pedido (solo posible en estado Pendiente), el stock de los productos del pedido original se devuelve y se descuenta nuevamente según las líneas nuevas, todo dentro de la misma operación atómica de *SaveChangesAsync()*. Si el stock disponible es insuficiente para cualquier línea, la operación se rechaza completamente y se informa al usuario.

4.5 Exportación de documentos (PDF y Excel)

Se implementaron dos tipos de exportación sin dependencias de servicio externo:

- **PDF de factura individual:** generado programáticamente con la biblioteca iText 7. Incluye encabezado con nombre del sistema, número de pedido, fecha, estado, datos del cliente, datos del vendedor y tabla de líneas con totales. El archivo se retorna en memoria como *FileResult* sin persistencia en disco.
- **Excel de listado de pedidos:** generado con EPPlus. Exporta el conjunto de pedidos filtrados (mismo conjunto visible en la tabla del índice) con encabezados formateados, filas alternadas y totales al pie. Permite al área administrativa realizar análisis fuera del sistema.

5.EVIDENCIAS DE REQUISITOS IMPLEMENTADOS

REQUISITO R-01 - CRUD DE PRODUCTOS

Catálogo de Productos

12 producto(s) encontrado(s)

Nombre

Categoría

Por página

Filtrar

Tecnología



Amazon Basics Monitor para jue...

€84 195,00

25

% IVA 13,00%

Activo

Detalle

Música



ARTPOP

€17 000,00

5

% IVA 10,00%

Activo

Detalle

Tecnología



ASUS ROG Xbox Ally Gaming po...

€250 000,00

10

% IVA 13,00%

Activo

Detalle

Música



Bad Vibes Forever

€14 000,00

10

% IVA 13,00%

Activo

Detalle

REQUISITO R-02 - CRUD DE CLIENTES

Clientes

+ Nuevo Cliente

3 cliente(s) registrado(s)

BUSCAR

Nombre

Cédula

Por página

Nombre del cliente...

9 dígitos...

10

Q Filtrar

X

NOMBRE

CÉDULA

CORREO

TELÉFONO

Jimena Chaves

148587981

jimena@gmail.com

22221010

Detalle

Justin Esquivel Marchena

11530354

Pruebajax@gmail.com

87785858

Detalle

Prueba

118345132

afjimenez@gmail.com

88101010

Detalle

REQUISITO R-03 - GESTIÓN DE PEDIDOS

Pedidos

+ Nuevo Pedido

Excel

6 pedido(s) encontrado(s)

FILTROS AVANZADOS

Estado

Fecha desde

Fecha hasta

Cliente

Vendedor

Por página

Todos

dd/mm/aaaa

dd/mm/aaaa

Nombre del cliente...

Nombre del vendedor...

10

Q

X

#

FECHA

CLIENTE

VENDEDOR

ESTADO

SUBTOTAL

IMPUESTOS

TOTAL

#6

18/03/2026
20:42

Justin Esquivel Marchena

Administrador

Facturado

€63 920,00

€4 149,60

€68 069,60

Ver

#5

18/03/2026
19:36

Prueba

Administrador

Confirmado

€64 820,00

€8 426,60

€73 246,60

Ver

#4

18/03/2026
19:30

Prueba

Administrador

Facturado

€191 800,00

€24 934,00

€216 734,00

Ver

#3

17/03/2026
20:59

Justin Esquivel Marchena

Administrador

Facturado

€59 136,00

€7 687,68

€66 823,68

Ver

#2

15/03/2026
18:42

Justin Esquivel Marchena

Administrador

Facturado

€101 800,00

€13 234,00

€115 034,00

Ver

#1

15/03/2026
18:22

Justin Esquivel Marchena

Administrador

Facturado

€37 800,00

€4 914,00

€42 714,00

Ver

Mostrando 6 de 6 pedidos · Página 1 de 1

REQUISITO R-04 - CONTROL DE ACCESO POR ROLES

```
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin")]
0 referencias
public async Task<IActionResult> Edit(int id, [Bind("ClienteId,Nombre,Cedula,Correo,Telefono")] Cliente cliente)
{
    if (id != cliente.ClienteId) return NotFound();

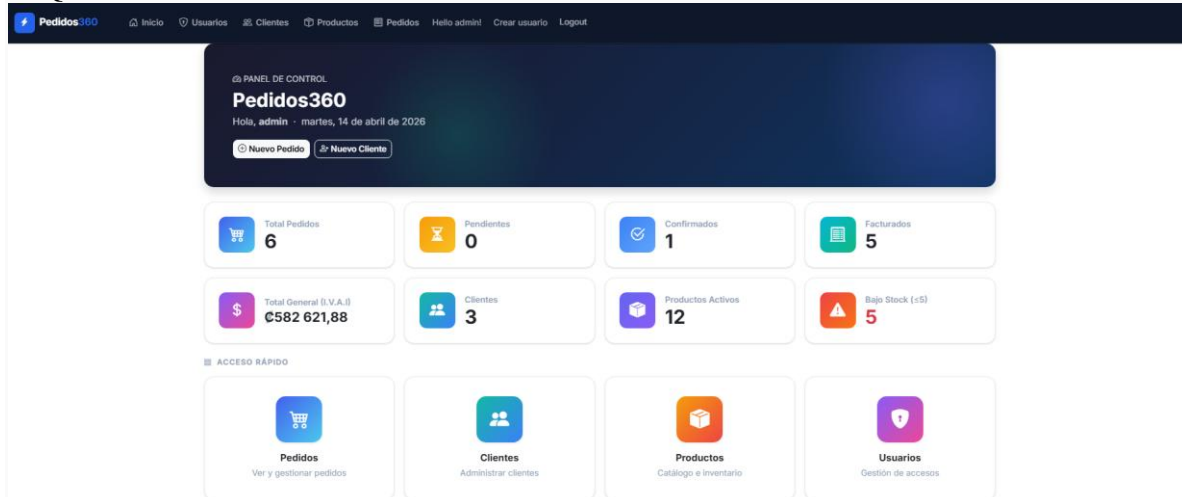
    if (!ModelState.IsValid)
        return AjaxModelStateErrorOrView(cliente);

    if (await _context.Clientes.AnyAsync(c => c.ClienteId != id && c.Cedula == cliente.Cedula))
        ModelState.AddModelError(nameof(Cliente.Cedula), "Ya existe otro cliente con esa cédula.");

    if (await _context.Clientes.AnyAsync(c => c.ClienteId != id && c.Correo == cliente.Correo))
        ModelState.AddModelError(nameof(Cliente.Correo), "Ya existe otro cliente con ese correo.");
    if (await _context.Clientes.AnyAsync(c => c.ClienteId != id && c.Nombre == cliente.Nombre))
        ModelState.AddModelError(nameof(Cliente.Nombre), "Ya existe otro cliente con ese nombre.");

    if (await _context.Clientes.AnyAsync(c => c.ClienteId != id && c.Telefono == cliente.Telefono))
        ModelState.AddModelError(nameof(Cliente.Telefono), "Ya existe otro cliente con ese número de teléfono.");
```

REQUISITO R-05 - DASHBOARD DE INDICADORES



REQUISITO R-06 - API REST INTERNA CON AJAX

```

v [🔒] [📁] Controllers
  v [🔒] [📁] Api
    > [🔒] [C#] CategoríasApiController.cs
    > [🔒] [C#] PedidosApiController.cs
    > [🔒] [C#] ProductosApiController.cs

```


6.CONCLUSIONES

El proyecto Pedidos360 cumple los objetivos planteados para el curso de Lenguajes de Bases de Datos. Se diseñó un esquema relacional normalizado que evita la redundancia de datos (los precios e impuestos se desnormalizan deliberadamente en PEDIDO_DETALLE para preservar el histórico transaccional) y se establecieron restricciones de integridad referencial coherentes con las reglas de negocio.

El uso de Entity Framework Core como capa de abstracción permitió gestionar las migraciones del esquema de forma incremental sin pérdida de datos, y garantizó que las operaciones complejas (crear pedido con múltiples detalles y actualización de stock simultánea) se ejecuten dentro de una única transacción atómica.

La incorporación de una API REST interna demuestra la posibilidad de combinar el patrón MVC con servicios de datos JSON dentro de la misma aplicación, habilitando experiencias de usuario más fluidas sin necesidad de un frontend independiente.

La arquitectura de roles implementada garantiza que la lógica de autorización esté centralizada en los atributos de los controladores y no dispersa en la lógica de presentación, lo que facilita la auditoría de seguridad y la extensión futura de permisos.

Como trabajo futuro se identifican las siguientes mejoras potenciales: implementación de autenticación multifactor (2FA), paginación del lado del servidor en los reportes de exportación, integración de una pasarela de pagos, y migración a una arquitectura de microservicios para escalar el módulo de pedidos de forma independiente.